

Understanding GitHub Actions

Learn the basics of core concepts and essential terminology in GitHub Actions.

Overview

GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline. You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.

GitHub Actions goes beyond just DevOps and lets you run workflows when other events happen in your repository. For example, you can run a workflow to automatically add the appropriate labels whenever someone creates a new issue in your repository.

GitHub provides Linux, Windows, and macOS virtual machines to run your workflows, or you can host your own self-hosted runners in your own data center or cloud infrastructure.

The components of GitHub Actions

You can configure a GitHub Actions **workflow** to be triggered when an **event** occurs in your repository, such as a pull request being opened or an issue being created. Your workflow contains one or more **jobs** which can run in sequential order or in parallel. Each job will run inside its own virtual machine **runner**, or inside a container, and has one or more **steps** that either run a script that you define or run an **action**, which is a reusable extension that can simplify your workflow.

![Diagram of an event triggering Runner 1 to run Job 1, which triggers Runner 2 to run Job 2. Each of the jobs is broken into multiple steps.](/assets/images/help/actions/overview-actions-simple.png)

Workflows

A **workflow** is a configurable automated process that will run one or more jobs. Workflows are defined by a YAML file checked in to your repository and will run when triggered by an event in your repository, or they can be triggered manually, or at a defined schedule.

Workflows are defined in the `.github/workflows` directory in a repository. A repository can have multiple workflows, each of which can perform a different set of tasks such as:

- * Building and testing pull requests
- * Deploying your application every time a release is created
- * Adding a label whenever a new issue is opened

You can reference a workflow within another workflow. For more information, see [Reuse workflows](/en/actions/using-workflows/reusing-workflows).

For more information, see [Writing workflows](/en/actions/using-workflows).

Events

An **event** is a specific activity in a repository that triggers a **workflow** run. For example, an activity can originate from GitHub when someone creates a pull request, opens an issue, or pushes a commit to a repository. You can also trigger a workflow to run on a [\[schedule\]](/en/actions/using-workflows/events-that-trigger-workflows#schedule), by [posting to a REST API](/en/rest/repos/repos#create-a-repository-dispatch-event), or manually.

For a complete list of events that can be used to trigger workflows, see [\[Events that trigger workflows\]](/en/actions/using-workflows/events-that-trigger-workflows).

Jobs

A **job** is a set of **steps** in a workflow that is executed on the same **runner**. Each step is either a shell script that will be executed, or an **action** that will be run. Steps are executed in order and are dependent on each other. Since each step is executed on the same runner, you can share data from one step to another. For example, you can have a step that builds your application followed by a step that tests the application that was built.

Steps run in order by default, but you can also run selected steps concurrently when your workflow benefits from parallel execution, such as starting a long-running service while later steps continue. For more information, see [\[Workflow syntax for GitHub Actions\]](/en/actions/reference/workflows-and-actions/workflow-syntax#jobsjob_idstepsbackround).

You can configure a job's dependencies with other jobs; by default, jobs have no dependencies and run in parallel. When a job takes a dependency on another job, it waits for the dependent job to complete before running.

You can also use a **matrix** to run the same job multiple times, each with a different combination of variables—like operating systems or language versions.

For example, you might configure multiple build jobs for different architectures without any job dependencies and a packaging job that depends on those builds. The build jobs run in parallel, and once they complete successfully, the packaging job runs.

For more information, see [\[Choosing what your workflow does\]](/en/actions/using-jobs).

Actions

An **action** is a pre-defined, reusable set of jobs or code that performs specific tasks within a **workflow**, reducing the amount of repetitive code you write in your workflow files. Actions can perform tasks such as:

- * Pulling your Git repository from GitHub
- * Setting up the correct toolchain for your build environment
- * Setting up authentication to your cloud provider

You can write your own actions, or you can find actions to use in your workflows in the GitHub Marketplace.

For more information on actions, see [\[Reusing automations\]\(/en/actions/creating-actions\)](/en/actions/creating-actions).

Runners

A **runner** is a server that runs your workflows when they're triggered. Each runner can run a single **job** at a time.

GitHub provides Ubuntu Linux, Microsoft Windows, and macOS runners to run your **workflows**. Each workflow run executes in a fresh, newly-provisioned virtual machine.

GitHub also offers larger runners, which are available in larger configurations. For more information, see [\[Using larger runners\]\(/en/actions/using-github-hosted-runners/using-larger-runners\)](/en/actions/using-github-hosted-runners/using-larger-runners).

If you need a different operating system or require a specific hardware configuration, you can host your own runners.

For more information about self-hosted runners, see [\[Managing self-hosted runners\]\(/en/actions/how-to/creating-self-hosted-runners\)](/en/actions/how-to/creating-self-hosted-runners).

Next steps

GitHub Actions can help you automate nearly every aspect of your application development processes. Ready to get started? Here are some helpful resources for taking your next steps with GitHub Actions:

- * To create a GitHub Actions workflow, see [\[Using workflow templates\]\(/en/actions/how-to/write-workflows/write-workflow-templates\)](/en/actions/how-to/write-workflows/write-workflow-templates).
- * For continuous integration (CI) workflows, see [\[Building and testing your code\]\(/en/actions/tutorials/build-and-test-code\)](/en/actions/tutorials/build-and-test-code).
- * For building and publishing packages, see [\[Publishing packages\]\(/en/actions/tutorials/publish-packages\)](/en/actions/tutorials/publish-packages).
- * For deploying projects, see [\[Deploying to third-party platforms\]\(/en/actions/how-to/deploy/deploy-to-third-party-platforms\)](/en/actions/how-to/deploy/deploy-to-third-party-platforms).
- * For automating tasks and processes on GitHub, see [\[Managing your work with GitHub Actions\]\(/en/actions/tutorials/manage-your-work\)](/en/actions/tutorials/manage-your-work).
- * For examples that demonstrate more complex features of GitHub Actions, see [\[Choosing what your workflow does\]\(/en/actions/how-to/write-workflows/choose-what-workflows-do\)](/en/actions/how-to/write-workflows/choose-what-workflows-do). These detailed examples explain how to test your code on a runner, access the GitHub CLI, and use advanced features such as concurrency and test matrices.
- * To certify your proficiency in automating workflows and accelerating development with GitHub Actions, earn a GitHub Actions certificate with GitHub Certifications. For more information, see [\[About GitHub Certifications\]\(/en/get-started/showcase-your-expertise-with-github-certifications/about-github-certifications\)](/en/get-started/showcase-your-expertise-with-github-certifications/about-github-certifications).

